

Section 1: Problem Understanding / Introduction

I began by checking what each row in the CSV represented. I was given a farm sustainability dataset in CSV format, and my task was to turn that data into a proper database system. The data contains 20 resource-application rows, covering 5 farms, 10 crop-production records, 5 soil profiles, 3 resource types and 5 sustainability initiatives. The CSV is structured, but it is still not a good database design because it mixes different entities together in one repeated table. One row represents a resource application linked to a crop grown on a farm. The same values are repeated across several rows, which can cause duplication and possible inconsistency. For example, South Farm, Kent appears in four CSV rows, so correcting that location would require four separate updates. A relational MySQL database is suitable because the data is structured and naturally made of connected entities. By separating the data into related tables, I reduced duplication, supported data integrity and made it easier to query sustainability information such as energy usage, crop yield and environmental impact.

Section 2: Relational Database Design and Rationale

I started by identifying the main entities because the CSV mixed several different things together. I used seven tables because each table has a clear purpose instead of keeping all values in one large repeated table.

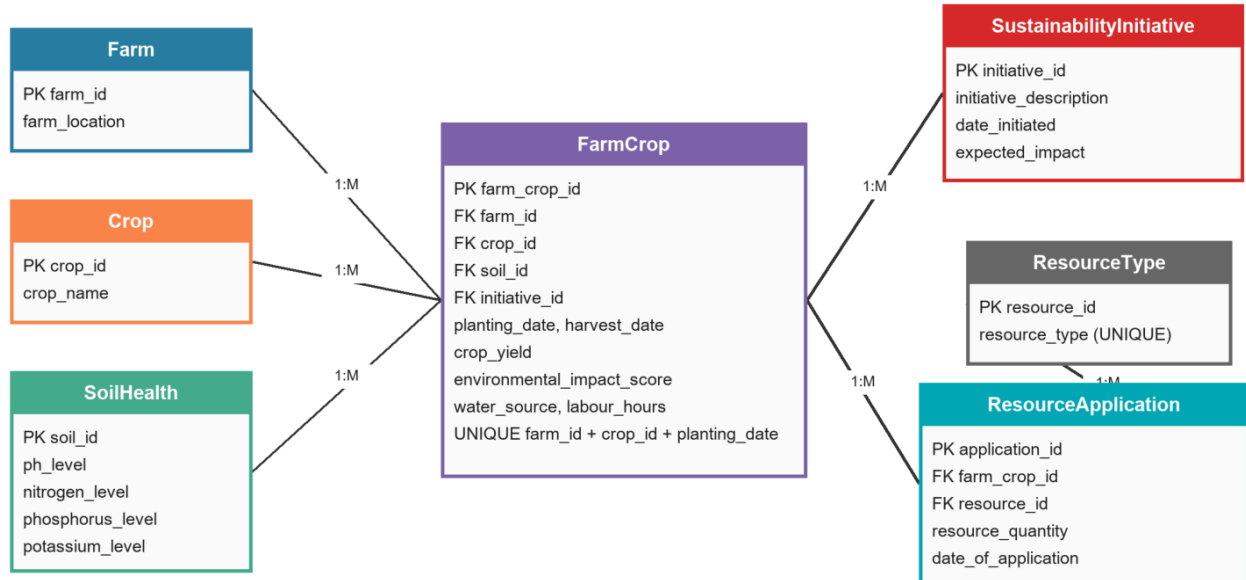
Table	Main key / important links	Purpose
Farm	farm_id	Stores each supplied farm identifier and farm_location once.
Crop	crop_id	Stores each crop once.
SoilHealth	soil_id	Stores pH, nitrogen, phosphorus and potassium values.
ResourceType	resource_id, resource_type UNIQUE	Stores controlled resources such as Water, Fertilizer and Energy.
SustainabilityInitiative	initiative_id	Stores each initiative description, date initiated and expected impact.
FarmCrop	farm_crop_id, FKs to Farm, Crop, SoilHealth and SustainabilityInitiative	Stores one crop-production record with dates, yield, score, water source and labour hours.
ResourceApplication	application_id, FKs to FarmCrop and ResourceType	Stores quantity and application date for each resource event.

The most important design choice is FarmCrop because the key business record is not just a farm or crop separately, but a specific crop grown on a specific farm. That is why planting date, harvest date, yield, environmental impact score, water source and labour hours belong in FarmCrop. ResourceApplication is separate because one FarmCrop record can have multiple resource applications, each with its own type, quantity and date. Soil profile and initiative are linked to FarmCrop because they may vary between crops or growing periods. In this small sample each farm reuses one soil profile and one initiative, but I do not treat that as a permanent rule. Resource units are not supplied, so I preserved the quantities exactly. I also avoided comparing Water, Fertilizer and Energy totals as if they used the same unit.

Section 3: Entity Relationship Diagram and Relationships

The ERD is based on the seven tables above. FarmCrop is placed in the centre because it connects a farm to a crop and stores the crop-production information. ResourceApplication is attached to FarmCrop because resources are applied to a specific crop-production record, not just to a farm or a crop separately. This also avoids a fan trap because every resource event links to one exact farm-crop production record.

Farm Sustainability Database ERD



Relationship	Multiplicity / participation
Farm - FarmCrop	Farm 0..*; FarmCrop 1..1. Every production record must reference exactly one farm.
Crop - FarmCrop	Crop 0..*; FarmCrop 1..1. Every production record must reference exactly one crop.
SoilHealth - FarmCrop	SoilHealth 0..*; FarmCrop 1..1 because soil_id is mandatory.
SustainabilityInitiative - FarmCrop	Initiative 0..*; FarmCrop 1..1 because initiative_id is mandatory.
FarmCrop - ResourceApplication	FarmCrop 0..*; ResourceApplication 1..1. A production record can have many resource applications.
ResourceType - ResourceApplication	ResourceType 0..*; ResourceApplication 1..1 because resource_id is mandatory.

Section 4: Normalisation and Design Justification

After looking at the CSV, I realised that one table with all the columns would not be a good design. The file repeats the same farm, crop, soil, resource and initiative information across many rows. This design follows normalisation. In 1NF, I avoided storing repeated resource values in one field, so Water and Energy are separate rows rather than grouped together. In 2NF, I moved attributes to the table they fully depend on. For example, farm_location depends on farm_id, crop_name depends on crop_id and resource_type depends on resource_id. The production values stay in FarmCrop because they describe the full farm-crop record. In 3NF, I removed indirect dependencies. For example, if resource_type was stored directly inside ResourceApplication, then application_id would determine resource_id and resource_id would determine resource_type, creating a transitive dependency. Separating ResourceType keeps the design cleaner and avoids repeating Energy, Water or Fertilizer in many application rows. Important functional dependencies include farm_id -> farm_location, crop_id -> crop_name, soil_id -> soil measurements, resource_id -> resource_type and initiative_id -> initiative details. I chose this instead of one large table because a single table would create update anomalies. I also kept resource data out of FarmCrop because one farm-crop record can have multiple resource applications.

Section 5: SQL Implementation and Validation

The SQL script uses DDL statements such as CREATE DATABASE, CREATE TABLE, named constraints, referential actions, indexes and a view to define the database structure. It then uses DML statements such as INSERT and SELECT to populate, validate and analyse the supplied data. MySQL DATE is used for all dates, DECIMAL(12,2) for quantities and AUTO_INCREMENT for generated IDs. Each table has a primary key for entity integrity, while foreign keys give referential integrity, for example FarmCrop.farm_id references Farm.farm_id and ResourceApplication.resource_id references ResourceType.resource_id. I also added NOT

NULL and CHECK constraints for values such as pH 0-14, non-negative quantities, impact score 1-5 and harvest_date occurring on or after planting_date. UNIQUE constraints are used for resource_type and the farm/crop/planting-date combination. I added indexes on the main filtering and join columns because the API and analysis queries search by farm, crop, resource type and application date. A joined view was also created for the farm sustainability profile, so the same multi-table result can be queried without rewriting the full join each time.

Validation queries confirmed 5 farms, 10 crops, 5 soil profiles, 3 resource types, 5 initiatives, 10 FarmCrop records, 20 ResourceApplication records, 10 Energy rows and total Energy quantity of 2,464,000. The joined query returned 20 rows, showing that the normalised tables can reconstruct the source information. The validation also showed two Energy applications after the recorded harvest date: Corn has Energy on 2023-09-24 after harvest on 2023-09-15, and Soybeans has Energy on 2023-10-12 after harvest on 2023-09-20. I kept these values because the task was to insert the supplied data, not silently change it. I also added a GROUP BY analysis query to compare total energy use and average environmental impact score by farm, showing that the database can support sustainability analysis.

Section 6: RESTful API Design

The database stores the data, but a RESTful API gives controlled routes for reading, adding, updating and deleting records. I designed the API around resources, not actions, so the route names are based on farms, farm-crops, resource applications, initiatives and energy usage. HTTP methods show the operation: GET reads, POST creates, PATCH or PUT updates and DELETE removes data. The API uses path parameters to identify records, query parameters to filter collections and JSON request bodies for create or update fields. I also included status codes such as 200 OK, 201 Created, 400 Bad Request, 404 Not Found and 500 Internal Server Error.

Method and route	Parameters / body	Purpose
GET /farms; POST /farms	GET query: location. POST body: farm_id, farm_location.	List, filter or create farms.
GET /farms/{farm_id}	Path: farm_id.	Return one farm sustainability profile.
GET /farm-crops; POST /farm-crops	GET query: farm_id, crop_id, initiative_id, score_min and score_max. POST body: FarmCrop fields.	Search or create production records.
GET/PATCH /farm-crops/{farm_crop_id}	Path: farm_crop_id. PATCH body: dates, yield, score, water_source or labour_hours.	Retrieve or partially update production.
GET /resource-applications; POST /resource-applications	GET query: farm_crop_id, resource_id, date_from and date_to. POST body: farm_crop_id, resource_id, quantity and date.	Search or create resource events.
GET/PATCH/DELETE /resource-applications/{application_id}	Path: application_id. PATCH body: quantity and/or date.	Retrieve, update or remove one event.
GET /energy-usage	Query: farm_id, crop_id, date_from and date_to.	Return energy events joined to yield and impact.
GET /initiatives; GET /initiatives/{initiative_id}	Optional query: description. Path: initiative_id.	List or retrieve initiatives.

For creating a resource application, POST /resource-applications could accept {"farm_crop_id":1, "resource_id":3, "resource_quantity":360000.00, "date_of_application":"2023-05-30"}. Both IDs must exist, quantity must be non-negative and the date must be valid. I chose this instead of action routes like /getFarmData because RESTful APIs should use clear resource paths and standard HTTP methods.

Section 7: Document-Based Alternative / MongoDB Comparison

In the relational design, the data is split into tables and connected using keys, but in a document model the data can be nested inside one farm document. My chosen alternative would be a hybrid document model: crop-production records and resource applications would be embedded because they are normally read with the farm

profile, while controlled resource types and initiatives could be reference collections if central definitions are needed. The image below shows the same idea using the Wheat record from South Farm.

```
{
  "_id": 1,
  "farm_location": "South Farm, Kent",
  "productions": [{
    "crop": { "crop_id": 101, "crop_name": "Wheat" },
    "planting_date": "2023-03-15",
    "harvest_date": "2023-08-15",
    "soil_health": {
      "soil_id": 1,
      "ph": 6.5,
      "nitrogen": 50,
      "phosphorus": 20,
      "potassium": 180
    },
    "initiative": {
      "initiative_id": 1,
      "description": "Organic Farming",
      "date_initiated": "2023-01-01",
      "expected_impact": "Increase in yield"
    },
    "crop_yield": 3000,
    "environmental_impact_score": 4,
    "water_source": "River",
    "labour_hours": 150,
    "resource_applications": [
      { "resource_type": "Water", "quantity": 1000, "date": "2023-03-10" },
      { "resource_type": "Energy", "quantity": 360000, "date": "2023-05-30" }
    ]
  }]
}
```

A MongoDB query could use dot notation to search inside nested fields, for example `db.farms.find({"productions.resource_applications.resource_type": "Energy"})`. This is useful for profile-style reads, but it can duplicate values such as Energy, Water, Fertilizer or Crop Rotation across documents. For example, if the Crop Rotation initiative text was embedded separately inside both Green Acres crop records, correcting that text would require updating each embedded copy. In MySQL, the same correction would be one update in SustainabilityInitiative.

Criterion	Relational MySQL design	Document alternative
Integrity and consistency	Primary keys, foreign keys, UNIQUE and CHECK constraints reduce invalid relationships and repeated definitions.	Embedding simplifies reads, but repeated crop, resource or initiative details need application or schema validation.
Performance and use case	Strong for joins, cross-farm comparison, filtering and analytical aggregation.	Strong for retrieving one complete farm profile without joins.
Flexibility and scalability	Controlled schema changes and clear dependencies.	Flexible nested fields and horizontal partitioning, but large embedded arrays and duplication must be managed.
Updates	Shared facts are updated once.	Embedded facts may require updates across documents unless references are used.
Best fit	Best for this structured dataset and consistent sustainability analysis.	Useful where profile-style reads dominate and metrics change often.

My final judgement is that MongoDB would be useful for flexible farm-profile views, but the relational MySQL design is more suitable here because the data is structured, repeated and analytical, and it needs stronger integrity control. MySQL is also stronger for queries such as total energy by crop or average environmental impact by initiative because SQL joins and GROUP BY queries are designed for this type of analysis.

Section 8: Conclusion

Overall, I converted the repeated CSV into a normalised MySQL database that reduces duplication, protects data integrity and supports sustainability analysis. The relational model is the stronger fit for this task because it supports reliable relationships, constraints and cross-farm analysis, while the document model is more useful for complete farm-profile retrieval.